

Giải pháp open-source cho video call 1:1 trên web

Kiến trúc tự triển khai và phương án kiểm chứng

Người thực hiện	Đặng Phúc An Khang	Bộ phận	Kỹ thuật
Vị trí	Thực tập sinh Full-stack Developer	Người giao và duyệt	CTO Tín Nguyễn
Thời gian thực hiện	04/08/2026 - 05/08/2026	Ngày báo cáo	05/08/2026

Mục tiêu và phạm vi

Báo cáo đánh giá giải pháp mã nguồn mở, self-host cho cuộc gọi video 1:1 trên web. Tiêu chí chính gồm quyền kiểm soát giao diện, vòng đời cuộc gọi, định danh, phân quyền, đường truyền media và dữ liệu vận hành. Dịch vụ managed-only bị loại khỏi tập ứng viên.

Phạm vi phiên bản đầu gồm lời mời gọi, chấp nhận, từ chối, hủy, bận, không trả lời, truyền âm thanh-hình ảnh và kết thúc cuộc gọi. Phạm vi mở rộng gồm recording, gọi nhóm, chia sẻ màn hình và thông báo khi trình duyệt đã đóng.

LiveKit self-host là phương án được đề xuất để tiếp tục kiểm chứng. mediasoup được dùng để đối chiếu khi cần điều khiển media ở mức thấp. Raw WebRTC kết hợp coturn là mốc đối chiếu cho kết nối 1:1 và TURN. Janus yêu cầu phê duyệt giấy phép GPL-3.0 trước khi thử nghiệm.

Feasibility PoC sử dụng LiveKit Server 1.13.1, backend ASP.NET Core, SignalR và frontend Vue 2. Kết quả smoke test ứng dụng và bảo mật đạt 15/15. Các mục đã xác nhận gồm tenant isolation, token theo room/identity, busy reservation, xử lý event lặp, HTTPS API và signaling WSS. Giao diện đạt kiểm tra tại viewport 360x800, 390x844 và trên iPhone thật.

Laptop và iPhone đã cùng vào phòng, cấp quyền và phát media; laptop nhận được video từ iPhone. Popup nhận cuộc gọi và màn hình gọi riêng đã được triển khai. Video dọc được giữ đúng tỷ lệ và không cắt khung hình. Các mục chưa nghiệm thu gồm A/V hai chiều sau bản sửa, Android, kết nối khác mạng, TURN, khôi phục kết nối, tải và vận hành. Trạng thái hiện tại: chưa phê duyệt production.

1 Thu nhận media và signaling

1.1 Camera và microphone

Trình duyệt truy cập camera và microphone bằng `getUserMedia()`. API yêu cầu HTTPS và quyền của người dùng. Kết quả trả về là một `MediaStream` chứa các `MediaStreamTrack`. Ứng dụng đưa các track này vào `RTCPeerConnection` để truyền qua WebRTC [1, 2].

Frontend quản lý quyền thiết bị, thiết bị đang chọn và trạng thái track. Backend không truy cập camera hoặc microphone. Các trường hợp bắt buộc xử lý gồm từ chối quyền, mất thiết bị, đổi camera, tắt track và trình duyệt chuyển sang background.

1.2 RTCPeerConnection và SDP

`RTCPeerConnection` quản lý luồng gửi, luồng nhận, ICE transport, codec và trạng thái kết nối. Hai phía trao đổi offer và answer chứa SDP. SDP mô tả loại media, codec, hướng truyền, ICE credential và tham số RTP/RTCP [3].

Trình tự tối thiểu gồm tạo peer connection, thêm local track, tạo offer, đặt local description, chuyển offer tới phía còn lại, tạo answer và trao đổi ICE candidate. Remote track được gắn vào phần tử media khi sự kiện track phát sinh.

1.3 Vai trò của signaling

WebRTC không cung cấp kênh signaling. Ứng dụng sử dụng WebSocket, SignalR hoặc HTTPS để trao đổi offer, answer và ICE candidate [4]. Kênh này cũng truyền các sự kiện cuộc gọi: incoming, accept, reject, cancel, busy, timeout và hangup.

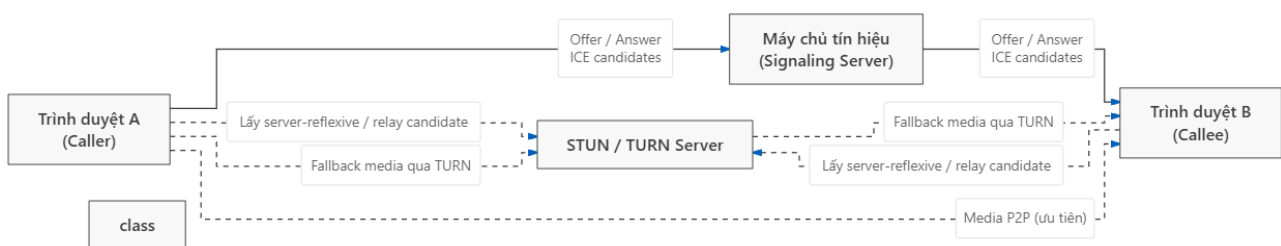
Media signaling và call-control signaling phải được phân loại rõ. Backend quản lý trạng thái nghiệp vụ. Event của trình duyệt phản ánh trạng thái kết nối tại từng endpoint.

2 ICE, STUN, TURN và topology media

2.1 NAT traversal

ICE tìm đường truyền qua NAT bằng các candidate [5]. Host candidate là địa chỉ cục bộ. Server-reflexive candidate là địa chỉ công khai do STUN phát hiện. Relay candidate là địa chỉ chuyển tiếp do TURN cấp. Hai phía trao đổi và kiểm tra các candidate qua signaling.

TURN chuyển tiếp media khi đường truyền trực tiếp thất bại [6]. Môi trường production yêu cầu TURN do hệ thống kiểm soát, credential có thời hạn, TLS và giám sát bằng thông.



Hình 1. Signaling, STUN/TURN và các đường truyền media trong WebRTC.

Kiểm thử TURN phải đặt `iceTransportPolicy=relay` hoặc xác nhận selected candidate pair có loại relay. WebRTC Statistics API cung cấp candidate pair, RTT, jitter, packet loss, bitrate và thông tin frame [7].

2.2 P2P và SFU

Trong mô hình P2P, hai trình duyệt truyền media trực tiếp. TURN tiếp nhận media khi P2P thất bại. Ứng dụng tự quản lý khôi phục kết nối, quan sát và tương thích trình duyệt.

Trong mô hình SFU, mỗi client gửi media tới máy chủ và nhận track do máy chủ chuyển tiếp. SFU tập trung việc điều phối track, quan sát, mở rộng và recording. Đổi lại, máy chủ phải xử lý băng thông media. Với cuộc gọi 1:1, quyết định dùng SFU dựa trên yêu cầu vận hành và mở rộng.

3 Vòng đời cuộc gọi trực tiếp

3.1 Ranh giới trách nhiệm

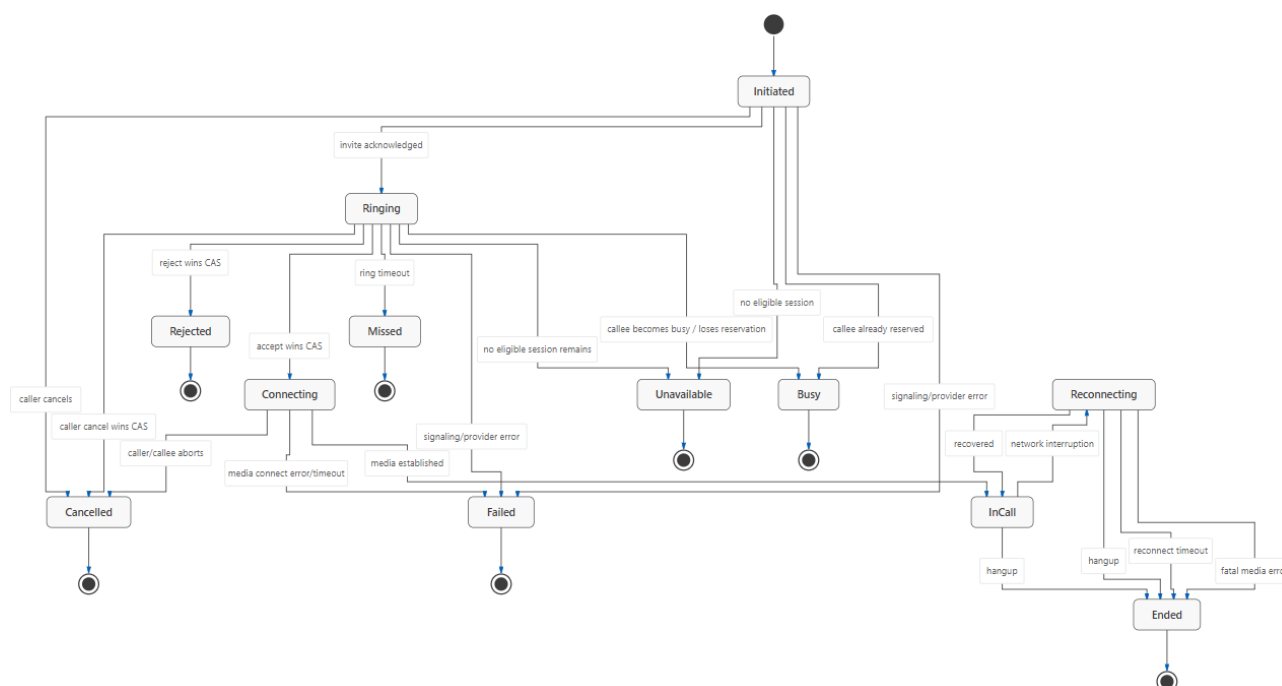
WebRTC xử lý kết nối media, không quản lý nghiệp vụ cuộc gọi [4]. Backend quản lý người gọi, người nhận, tenant, trạng thái bận, thời gian chờ chuông và `CallSession`. Media room hoặc peer connection biểu diễn phiên truyền thông.

Thành phần	Trách nhiệm
Backend ứng dụng	Xác thực, tenant, quyền gọi, presence, busy reservation, trạng thái và call log
Kênh realtime	Phân phối incoming call, accept, reject, cancel và terminal event
Nền tảng media	Signaling WebRTC, transport, SFU/gateway, track và thống kê media
STUN/TURN	NAT traversal và relay khi đường trực tiếp không khả dụng
Frontend	Thiết bị, giao diện cuộc gọi, hiển thị state và remote media

Bảng 1. Ranh giới trách nhiệm của module video call.

3.2 State machine và race condition

State machine gồm Initiated, Ringing, Connecting, InCall, Reconnecting và các trạng thái kết thúc. Accept, reject, cancel và timeout được ghi bằng chuyển trạng thái nguyên tử. Với nhiều tab hoặc thiết bị, backend chỉ chấp nhận một thao tác accept. Event lặp hoặc đến muộn không được thay đổi terminal state.



Hình 2. State machine của cuộc gọi trực tiếp.

Backend cấp media credential sau khi xác nhận người nhận, tenant và trạng thái accept. Token có thời hạn ngắn, giới hạn theo room, identity và quyền publish/subscribe. Event của media server phục vụ đối soát. Cơ sở dữ liệu vẫn lưu trạng thái nghiệp vụ chính thức.

3.3 Presence và nhiều session

Presence cho biết session đang có khả năng nhận signaling. Backend dùng busy reservation riêng để khóa trạng thái bận. Khi không còn session hợp lệ, cuộc gọi chuyển sang Unavailable. Khi hết thời gian đồ chuông, cuộc gọi chuyển sang NoAnswer hoặc Missed.

4 Sàng lọc giải pháp triển khai

Điều kiện khảo sát gồm mã nguồn và giấy phép công khai, khả năng self-host signaling và media, hỗ trợ WebRTC trên web và không phụ thuộc cloud của nhà cung cấp. Hoạt động của kho mã nguồn phản ánh trạng thái duy trì. Độ ổn định production cần kết quả qualification và dữ liệu vận hành.

Giải pháp	Giấy phép	Loại giải pháp	Kết quả sàng lọc
LiveKit	Apache-2.0	Programmable SFU	Chọn cho PoC ưu tiên
mediasoup	ISC	SFU cấp thấp	Giữ làm phương án đối chứng
Raw WebRTC + coturn	Chuẩn WebRTC; coturn mã nguồn mở	P2P/TURN tự xây	Giữ làm baseline kiến trúc
Janus	GPL-3.0	WebRTC gateway và plugin	Có điều kiện; cần duyệt giấy phép
Jitsi Meet	Apache-2.0	Nền tảng hội nghị	Không chọn; mô hình phòng họp
OpenVidu Community	Apache-2.0 cho thành phần cộng đồng	Phân phối tương thích LiveKit	Không chọn; thêm lớp phân phối và ranh giới Community/PRO
Element Call	AGPL-3.0	Ứng dụng MatrixRTC	Không chọn; phụ thuộc Matrix homeserver và LiveKit
MiroTalk SFU	AGPL-3.0	Ứng dụng hội nghị trên mediasoup	Không chọn; sản phẩm hoàn chỉnh
Nextcloud Talk	AGPL-3.0	Ứng dụng giao tiếp Nextcloud	Không chọn; phụ thuộc hệ sinh thái Nextcloud
Pion WebRTC	MIT	Thư viện WebRTC cho Go	Loại khỏi nhóm giải pháp module

Bảng 2. Kết quả sàng lọc giải pháp.

Jitsi gồm web app, Jitsi Videobridge, Jicofo và Prosody [8, 9]. OpenVidu Community phân phối API tương thích LiveKit; multi-node và high availability thuộc edition thương mại [10]. Element Call sử dụng MatrixRTC và LiveKit. MiroTalk và Nextcloud Talk là ứng dụng giao tiếp hoàn chỉnh [11, 12, 13]. Pion cung cấp WebRTC API ở cấp thư viện để xây media server [14].

Stringee và các API truyền thông theo mô hình managed service không đạt điều kiện self-host và bị loại khỏi ma trận PoC.

5 LiveKit và mediasoup

5.1 LiveKit

LiveKit là SFU mã nguồn mở theo giấy phép Apache-2.0. Nền tảng cung cấp SDK trình duyệt, room, participant, track, JWT grants, server API và webhook [15, 16]. Hệ thống có thể self-host toàn bộ media server. Token chứa identity, room và quyền publish/subscribe; backend ký token bằng API secret [17].

Frontend sử dụng JavaScript SDK và tự xây giao diện. Backend quản lý direct-call lifecycle. Room Service sử dụng HTTP/Twirp; backend có thể gọi API mà không cần SDK chính thức [18]. Recording yêu cầu Egress riêng và Redis khi tự host [19]. Core PoC không bao gồm recording.

LiveKit quản lý room và media. Backend SimlyDent quản lý incoming, busy và no-answer theo user. LiveKit chưa có server SDK .NET chính thức. PoC đã xác nhận backend .NET tự phát được JWT HS256 đúng room, identity và permission grant. Room Service chưa được sử dụng.

5.2 mediasoup

mediasoup là SFU cấp thấp theo giấy phép ISC. Server có dạng Node.js module hoặc Rust crate [20]. Ứng dụng tự quản lý signaling, protocol, authorization, reconnect, worker lifecycle và quan sát.

Trong hệ thống sử dụng backend .NET, mediasoup cần một dịch vụ Node.js hoặc Rust riêng. Điều kiện lựa

chọn mediasoup là yêu cầu kiểm soát media thấp tầng chưa được LiveKit đáp ứng. Chi phí bổ sung gồm phát triển, triển khai và vận hành dịch vụ media riêng.

Tiêu chí	LiveKit	mediasoup
Mức trừu tượng	SFU và SDK cấp sản phẩm	SFU cấp thấp
Direct-call state	Tự xây	Tự xây
Phù hợp backend .NET	JWT đã kiểm chứng; Twirp chưa dùng	Cần Node.js/Rust sidecar
Khối lượng media engineering	Thấp hơn	Cao hơn
Vai trò đề xuất	Phương án ưu tiên tiếp tục kiểm chứng	Đối chứng có điều kiện

Bảng 3. So sánh LiveKit và mediasoup.

6 Janus, Jitsi và Raw WebRTC

6.1 Raw WebRTC kết hợp coturn

Raw WebRTC sử dụng API trình duyệt, SignalR hoặc WebSocket cho signaling và coturn cho STUN/TURN [2, 21]. Ứng dụng kiểm soát trực tiếp SDP, ICE, relay và thông kê. Media 1:1 ưu tiên đường P2P khi mạng cho phép.

Ứng dụng phải tự triển khai glare handling, renegotiation, ICE restart, reconnect, device switching, credential TURN, quan sát, tương thích trình duyệt và recording. Vai trò hiện tại của Raw WebRTC là baseline PoC. Điều kiện production gồm năng lực phát triển WebRTC và vận hành hạ tầng truyền thông nội bộ.

6.2 Janus

Janus là WebRTC gateway theo giấy phép GPL-3.0. VideoCall plugin chuyển tiếp media giữa hai peer và gọi theo username [22, 23]. Backend vẫn phải quản lý user, tenant, quyền và trạng thái cuộc gọi. Điều kiện PoC: phê duyệt nghĩa vụ GPL-3.0.

6.3 Jitsi Meet

Jitsi Meet là nền tảng hội nghị mã nguồn mở theo giấy phép Apache-2.0. Nền tảng hỗ trợ self-host và nhúng qua IFrame. Kiến trúc và API tổ chức quanh conference room [8, 9]. Luồng gọi A-B theo user cần thêm lớp lời mời và state. Tùy biến sâu yêu cầu duy trì fork của toàn bộ ứng dụng hội nghị.

Jitsi phục vụ mô hình phòng họp. Kết quả sàng lọc: không chọn cho PoC gọi trực tiếp theo user và giao diện do SimlyDent sở hữu.

Tiêu chí	Raw WebRTC + coturn	Janus	Jitsi
Mô hình	P2P/TURN tự xây	Gateway/plugin	Conference platform
UI tùy biến	Toàn quyền	Toàn quyền	IFrame hoặc fork
Direct-call lifecycle	Tự xây	Plugin mẫu; production state tự xây	Tự xây ngoài meeting room
Điều kiện chính	Năng lực WebRTC nội bộ	Chấp nhận GPL-3.0	Chấp nhận meeting UX
Vai trò	Baseline	Có điều kiện	Không chọn cho core PoC

Bảng 4. Vai trò của Raw WebRTC, Janus và Jitsi.

7 Kết quả feasibility PoC

7.1 Phạm vi triển khai

PoC gồm LiveKit Server 1.13.1, backend ASP.NET Core, SignalR và frontend Vue 2. Docker Compose đóng gói toàn bộ stack. Backend quản lý tenant, lời mời, trạng thái cuộc gọi, busy reservation và token. LiveKit quản lý signaling WebRTC và media room.

State machine gồm Ringing, Accepted, Rejected, Cancelled và Ended. Backend cấp token LiveKit sau trạng thái Accepted. Token có TTL 5 phút và giới hạn theo room, identity. API secret nằm tại backend và LiveKit.

Giao diện hiện có danh sách người dùng và nút gọi. Phía nhận có popup chấp nhận hoặc từ chối. Sau khi chấp nhận, hai phía mở màn hình /call/{callId} riêng với video từ xa, video cục bộ và các nút bật/tắt camera, microphone, kết thúc cuộc gọi. Video dọc được giữ đúng tỷ lệ, co theo chiều cao vùng gọi và không cắt khung hình.

7.2 Kết quả kiểm tra

Hạng mục	Kết quả	Phạm vi bằng chứng
Build và khởi động stack	Đạt	Các service chính trong Docker Compose đang chạy
Application/security smoke	15/15 đạt	Tenant isolation, token scope, busy, duplicate transition và refresh ringing
HTTPS API qua Quick Tunnel	Đạt	UI và API trả HTTP 200; URL tunnel không cố định
LiveKit signaling WSS	Đạt	WebSocket kết nối và nhận frame signaling đầu tiên
Responsive và iPhone	Đạt	Viewport 360x800, 390x844; đã đổi chiều trên iPhone thật
Media thiết bị thật	Một phần	Hai phía phát media; laptop nhận video iPhone; chưa nghiệm thu hai chiều
Kết nối khác mạng	Chưa kiểm tra	LiveKit hiện quảng bá IP LAN của host
Forced TURN/relay	Chưa kiểm tra	Chưa triển khai public host và TURN
Reconnect, tải và raw stats	Chưa kiểm tra	Chưa thu selected candidate và dữ liệu vận hành

Bảng 5. Bằng chứng feasibility PoC tại thời điểm báo cáo.

Cloudflare Quick Tunnel cung cấp HTTPS và WebSocket cho phiên demo. Laptop và iPhone đã cùng vào một phòng LiveKit qua mạng Windows Mobile Hotspot. Server ghi nhận cả hai phía phát audio và video; laptop nhận được video từ iPhone. Client đã được bổ sung đăng ký chủ động luồng nhận, gắn lại track có sẵn và inline playback cho iOS. Chưa có lần nghiệm thu cuối cho A/V hai chiều sau bản sửa nên hạng mục media vẫn được đánh dấu một phần.

Phương án production yêu cầu domain, TLS và media endpoint cố định. Media server hiện quảng bá địa chỉ LAN; bằng chứng cuộc gọi khác mạng chưa có. Kiểm tra responsive hiện giới hạn ở viewport 360x800, 390x844 và một iPhone; Android chưa được kiểm tra.

7.3 Mức độ kiểm chứng

PoC đã xác nhận trạng thái cuộc gọi trực tiếp, tích hợp backend .NET, token scope, tenant isolation và khả năng phát media giữa laptop với iPhone trong cùng mạng thử nghiệm. LiveKit tiếp tục là phương án ưu tiên. Các mục còn mở gồm nghiệm thu A/V hai chiều, Android, kết nối khác mạng, TURN, khôi phục kết nối, tải, giám sát và vận hành.

8 Kiến trúc đề xuất và kết luận

8.1 Kiến trúc LiveKit self-host

Backend .NET quản lý CallSession, tenant, quyền, busy reservation và call log. SignalR hoặc WebSocket truyền lời mời và trạng thái. Sau khi ghi nhận accept, backend phát token LiveKit ngắn hạn theo room và identity. Hai client sau đó kết nối vào media room.

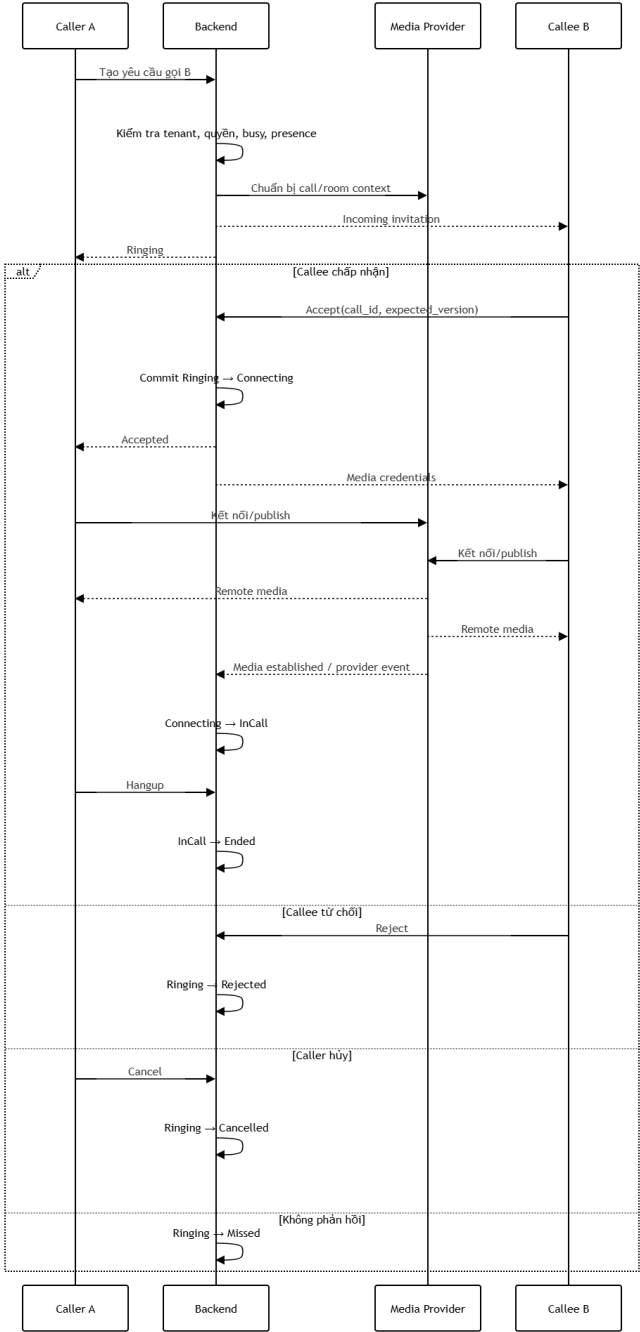
LiveKit room thiết lập kết nối media; LiveKit event phục vụ đối soát. PostgreSQL lưu trạng thái nghiệp vụ chính thức. TURN chạy trên hạ tầng doanh nghiệp và sử dụng credential có thời hạn.

8.2 Các bước kiểm chứng tiếp theo

1. Triển khai public domain, chứng thư TLS, media ports và TURN/TLS; xác nhận kết nối khác mạng và relay-only bằng selected candidate statistics.
2. Kiểm tra A/V hai chiều trên desktop, Chrome Android và Safari iOS bằng thiết bị thật; thực hiện no-answer, multi-tab, reconnect và thay đổi mạng.
3. Thay identity giả lập bằng authentication của hệ thống; lưu call state và call log trong PostgreSQL; bổ sung webhook reconciliation, timeout và rate limiting.
4. Thu thập CPU, RAM, băng thông, chất lượng media, TURN ratio và lỗi kết nối theo workload đã được phê duyệt.
5. Chỉ thử nghiệm mediasoup khi LiveKit không đáp ứng một yêu cầu kiểm soát media đã xác định. Raw WebRTC được dùng làm mốc đối chiếu cho kết nối 1:1 và TURN.

8.3 Kết luận

1. WebRTC xử lý kết nối và truyền media. Backend SimlyDent quản lý user, tenant, lời mời, trạng thái và call log.
2. LiveKit self-host là ứng viên ưu tiên. Tích hợp .NET qua JWT đã được kiểm chứng. Room Service chưa được sử dụng.
3. mediasoup dành cho yêu cầu điều khiển media cấp thấp. Raw WebRTC kết hợp coturn giữ vai trò mốc đối chiếu kỹ thuật.
4. Janus yêu cầu phê duyệt GPL-3.0. Jitsi và các ứng dụng hội nghị hoàn chỉnh bị loại khỏi PoC hiện tại.
5. PoC đã xác nhận trạng thái ứng dụng, tenant isolation, token scope và phát media giữa laptop với iPhone trong cùng mạng thử nghiệm. Nghiệm thu A/V hai chiều, Android, khác mạng, TURN, khôi phục kết nối, tải và vận hành vẫn còn mở.
6. Trạng thái hiện tại: chưa phê duyệt production.



Hình 3. Trình tự xử lý cuộc gọi trực tiếp và thiết lập media.

Tài liệu tham khảo

- [1] W3C. *Media Capture and Streams*. URL: <https://www.w3.org/TR/mediacapture-streams/> (truy cập ngày 05/08/2026).
- [2] W3C. *WebRTC: Real-Time Communication in Browsers*. URL: <https://www.w3.org/TR/webrtc/> (truy cập ngày 05/08/2026).
- [3] IETF. *RFC 9429: JavaScript Session Establishment Protocol*. URL: <https://www.rfc-editor.org/rfc/rfc9429.html> (truy cập ngày 05/08/2026).
- [4] IETF. *RFC 8825: Overview: Real-Time Protocols for Browser-Based Applications*. URL: <https://www.rfc-editor.org/rfc/rfc8825.html> (truy cập ngày 05/08/2026).
- [5] IETF. *RFC 8445: Interactive Connectivity Establishment*. URL: <https://www.rfc-editor.org/rfc/rfc8445.html> (truy cập ngày 05/08/2026).
- [6] IETF. *RFC 8656: Traversal Using Relays around NAT*. URL: <https://www.rfc-editor.org/rfc/rfc8656.html> (truy cập ngày 05/08/2026).
- [7] W3C. *WebRTC Statistics API*. URL: <https://www.w3.org/TR/webrtc-stats/> (truy cập ngày 05/08/2026).
- [8] Jitsi. *Jitsi Meet Repository*. URL: <https://github.com/jitsi/jitsi-meet> (truy cập ngày 05/08/2026).
- [9] Jitsi. *Jitsi Meet Architecture*. URL: <https://jitsi.github.io/handbook/docs/architecture/> (truy cập ngày 05/08/2026).
- [10] OpenVidu. *OpenVidu Platform Documentation*. URL: <https://openvidu.io/latest/docs/> (truy cập ngày 05/08/2026).
- [11] Element. *Element Call Repository*. URL: <https://github.com/element-hq/element-call> (truy cập ngày 05/08/2026).
- [12] MiroTalk. *MiroTalk SFU Repository*. URL: <https://github.com/miroslavpejic85/mirotalksfu> (truy cập ngày 05/08/2026).
- [13] Nextcloud. *Nextcloud Talk Repository*. URL: <https://github.com/nextcloud/spreed> (truy cập ngày 05/08/2026).
- [14] Pion. *Pion WebRTC Repository*. URL: <https://github.com/pion/webrtc> (truy cập ngày 05/08/2026).
- [15] LiveKit. *LiveKit Server Repository*. URL: <https://github.com/livekit/livekit> (truy cập ngày 05/08/2026).
- [16] LiveKit. *Self-hosting overview*. URL: <https://docs.livekit.io/transport/self-hosting/> (truy cập ngày 05/08/2026).
- [17] LiveKit. *Tokens and grants*. URL: <https://docs.livekit.io/home/server/generating-tokens> (truy cập ngày 05/08/2026).
- [18] LiveKit. *Room service API*. URL: <https://docs.livekit.io/reference/other/roomservice-api/> (truy cập ngày 05/08/2026).
- [19] LiveKit. *Self-hosted Egress service*. URL: <https://docs.livekit.io/transport/self-hosting/egress/> (truy cập ngày 05/08/2026).
- [20] mediasoup. *mediasoup Repository*. URL: <https://github.com/versatica/mediasoup> (truy cập ngày 05/08/2026).
- [21] coturn. *coturn TURN server Repository*. URL: <https://github.com/coturn/coturn> (truy cập ngày 05/08/2026).
- [22] Meetecho. *Janus WebRTC Server Repository*. URL: <https://github.com/meetecho/janus-gateway> (truy cập ngày 05/08/2026).
- [23] Meetecho. *Janus VideoCall plugin*. URL: <https://janus.conf.meetecho.com/docs/video-call> (truy cập ngày 05/08/2026).